JavaScript cheatsheet · 19 chapters · only JavaScript code

# Selenium with JavaScript: cheatsheet

The key ideas, every annotated line of every example in JavaScript, the tables and the common mistakes from every chapter, in one place for quick revision. Everything here comes from the chapters and shows JavaScript only, so this sheet is all you need. Each card links back to the lesson that teaches it.

93 key ideas | 51 code examples | 9 numbers | 61 tables | 39 tips and traps

## STEP 1 Meet Selenium

### 1.1 What Selenium is and where it came from

What Selenium does, its three parts, the key words, its history to the W3C standard and a first real program.

KEY IDEAS

- **What Selenium is** An umbrella project of tools and libraries that automate real browsers the way a user would. At its core, a browser user agent library.
- **Three parts** WebDriver (library plus browser driver, for code), IDE (record and play back extension), Grid (many sessions on many machines).
- **Not a test framework** Selenium drives the browser. Assertions, test structure and reports come from JUnit, pytest, Mocha and similar tools.
- **Key dates** Selenium 2.0 on 8 July 2011, 3.0 on 13 October 2016, W3C Recommendation on 5 June 2018, Selenium 4.0 on 13 October 2021.
- **Program shape** Start a session, send the browser somewhere, ask or act, and always quit at the end.

CODE

**A first look: open a page, read its title, close the browser**

| | |
|---|---|
| `const { Builder, Browser } = require('selenium-webdriver');` | Load the two pieces we need from the selenium-webdriver package. |
| `(async function firstLook() {` | Every Selenium call in JavaScript returns a promise, so we work inside an async function. |
| `const driver = await new Builder().forBrowser(Browser.CHROME).build();` | Build a driver for Chrome. Selenium Manager finds chromedriver and a session starts. |
| `await driver.get('https://www.selenium.dev/selenium/web/web-form.html');` | Open the page and wait for it to load. |
| `console.log('Title: ' + await driver.getTitle());` | Ask for the title. await waits for the browser to answer. |
| `console.log('URL: ' + await driver.getCurrentUrl());` | Ask which address the browser shows. |
| `await driver.quit();` | End the session and close the browser. |

Lesson: A first look at real code

TABLES

### Table: what people use selenium for · an everyday example

| What people use Selenium for | An everyday example |
|---|---|
| Testing web applications | Before a bank releases a new version of its login page, a test opens the page in Chrome, Firefox and Edge, logs in with a test account and checks the balance appears. |
| Repetitive tasks | A script logs into a supplier portal every morning at 6 and downloads the order report for the day. |
| Collecting data from pages | A researcher reads prices from a site that has no API. The docs ask you to check the terms of service of the site first, because some sites forbid it and some block Selenium. |

Lesson: What Selenium is

### Table: part · what it is · use it when

| Part | What it is | Use it when |
|---|---|---|
| **WebDriver** | A library for your programming language plus a driver for each browser. Your code sends commands, the driver makes the browser carry them out. | You write automated tests or scripts in Java, Python, JavaScript, C# or Ruby. This is what most of this subject teaches. |
| **IDE** | A Chrome and Firefox extension that records what you do in the browser and plays it back as Selenium commands. | You want a quick recording, or you are learning which command matches which action. |
| **Grid** | A server that spreads WebDriver sessions over many machines, browsers and operating systems. | One machine is too slow, or you must test on browser and operating system pairs you do not have. |

Lesson: WebDriver, IDE and Grid

### Table: word · meaning in the selenium documentation

| Word | Meaning in the Selenium documentation |
|---|---|
| API | The set of commands you use to control WebDriver, such as get, findElement and click. |
| Library | The code module that holds those commands for one language: a .jar file for Java, a pip package for Python, an npm package for JavaScript. |
| Driver | The program that controls one browser, such as chromedriver for Chrome and geckodriver for Firefox. Browser vendors write most of them. It usually runs on the same machine as the browser. |
| Framework | An extra library that supports a test suite: test runners such as JUnit, pytest or Mocha, or tools such as Cucumber. |

Lesson: WebDriver, IDE and Grid

### Table: version · released · what changed

| Version | Released | What changed |
|---|---|---|
| Selenium 2.0 | 8 July 2011 | The WebDriver APIs became the heart of Selenium. Selenium 1 code still ran. |
| Selenium 3.0 | 13 October 2016 | The original Selenium Core implementation was removed. Every major browser vendor now ships its own driver. |
| W3C WebDriver | 5 June 2018 | WebDriver became a W3C Recommendation, a standard every browser can implement the same way. |
| Selenium 4.0 | 13 October 2021 | Support for the old JSON Wire Protocol was removed. Only the W3C protocol is used. Relative locators and a rebuilt Grid arrived. |

Lesson: How Selenium grew

### TIPS AND TRAPS

**COMMON MISTAKE** Selenium only controls the browser. It does not decide whether a test passed, it does not write reports and it does not give you Given, When, Then syntax. Those come from a test framework such as JUnit, pytest or Mocha. Chapter 5.1 joins the two.

Lesson: What Selenium is

**REMEMBER** Always end with quit. If a program stops without it, the driver process and the browser keep running in the background and hold memory and ports. On a busy test machine those leftovers pile up quickly.

Lesson: A first look at real code

## 1.2 Architecture and the W3C WebDriver protocol

How a command travels from your code to the browser over the W3C WebDriver protocol, shown with real HTTP traffic.

### KEY IDEAS

- **Three programs** Your code and the binding (local end), the driver (remote end), the browser. The driver is written by the browser vendor.
- **HTTP and JSON** Every command is one HTTP request with a fixed method and path. Answers are {"value": ...} on success or {"value": {"error", "message", "stacktrace"}} on failure.
- **Session id** POST /session returns a session id. Every later path contains it. DELETE /session/{id} ends it.
- **Element reference** Elements travel as {"element-6066-11e4-a52e-4f735466cecf": "id"}. Bindings turn id, name and class name into CSS selectors.
- **BiDi** Classic WebDriver is request and response. WebDriver BiDi adds a WebSocket so the browser can send events. CDP access is temporary until BiDi covers it.

### CODE

**Drive Chrome with plain HTTP and no Selenium library at all**

| | |
|---|---|
| `const { spawn } = require('child_process');` | For starting the chromedriver program. No Selenium package is loaded. |
| `async function call(method, path, body) {` | Send one W3C command and return the part of the answer under "value". |
| `const res = await fetch('http://localhost:9515' + path, {` | fetch is built into Node.js 18 and later. |
| `body: body === undefined ? undefined : JSON.stringify(body),` | POST bodies are JSON objects. GET and DELETE have none. |
| `console.log(method, path, ' → ', text.length > 70 ? text.slice(0, 70) + '...' : text);` | Print the request and the start of the answer. |
| `const driver = spawn('chromedriver', ['--port=9515'], { stdio: 'ignore' });` | Start the driver yourself and hide its own messages. |
| `await new Promise(r => setTimeout(r, 1000));` | Give it a moment to open its port. |
| `const KEY = 'element-6066-11e4-a52e-4f735466cecf';` | The fixed key for element references. |
| `const s = '/session/' + (await call('POST', '/session', { capabilities: { alwaysMatch: { browserName: 'chrome' } } })).sessionId;` | New Session. |
| `await call('POST', s + '/timeouts', { implicit: 2000 });` | Set Timeouts: Find Element may keep looking for 2 seconds while the next page loads. |
| `await call('POST', s + '/url', { url: 'https://www.selenium.dev/selenium/web/web-form.html' });` | Navigate To. |
| `await call('GET', s + '/title');` | Get Title. |

| | |
|---|---|
| `const box = (await call('POST', s + '/element', { using: 'css selector', value: "[name='my-text']" }))[KEY];` | Find Element. |
| `await call('POST', s + '/element/' + box + '/value', { text: 'Selenium' });` | Element Send Keys. |
| `await call('POST', s + '/element/' + button + '/click', {});` | Element Click. |
| `await call('GET', s + '/element/' + message + '/text');` | Get Element Text. |
| `await call('DELETE', s);` | Delete Session. |
| `driver.kill();` | Stop chromedriver. |

Lesson: See the wire for yourself

## NUMBERS TO REMEMBER

**Success** Exam favourite

HTTP 200 and {"value": result}

The result is null for commands that only act, a string for text and titles, a list for Find Elements.

Lesson: The W3C WebDriver protocol

**Error** Exam favourite

HTTP 4xx or 5xx and {"value": {"error", "message", "stacktrace"}}

error is a fixed code such as no such element. The binding turns it into an exception.

Lesson: The W3C WebDriver protocol

**Element reference**

{"element-6066-11e4-a52e-4f735466cecf": "f.C6…e.6"}

A web element travels as an object with this one fixed key. The value is an opaque id made by the driver.

Lesson: The W3C WebDriver protocol

## TABLES

**Table: spec word · in practice · example**

| Spec word | In practice | Example |
|---|---|---|
| Local end | The client side, usually a language library on top of the protocol | selenium for Python, selenium-java, selenium-webdriver for npm |
| Remote end | The server side that carries out the commands | chromedriver, or Selenium Grid |
| Intermediary node | A remote end that passes commands on to another remote end | Selenium Grid, which forwards to a node |
| Endpoint node | The last remote end in the chain, built into or next to the browser | chromedriver driving Chrome |

Lesson: The pieces between your code and the browser

**Table: command · method · path**

| Command | Method | Path |
|---|---|---|
| New Session | POST | /session |
| Delete Session | DELETE | /session/{session id} |
| Navigate To | POST | /session/{id}/url |
| Get Title | GET | /session/{id}/title |
| Find Element | POST | /session/{id}/element |
| Find Elements | POST | /session/{id}/elements |
| Element Click | POST | /session/{id}/element/{element id}/click |
| Element Send Keys | POST | /session/{id}/element/{element id}/value |
| Get Element Text | GET | /session/{id}/element/{element id}/text |
| Set Timeouts | POST | /session/{id}/timeouts |
| Execute Script | POST | /session/{id}/execute/sync |
| Take Screenshot | GET | /session/{id}/screenshot |

Lesson: The W3C WebDriver protocol

**Table: error code · http status**

| Error code | HTTP status |
|---|---|
| no such element | 404 |
| stale element reference | 404 |
| invalid session id | 404 |
| element not interactable | 400 |
| element click intercepted | 400 |
| invalid selector | 400 |
| timeout | 500 |
| unexpected alert open | 500 |
| session not created | 500 |

Lesson: The W3C WebDriver protocol

**Table: layout · where things run · code difference**

| Layout | Where things run | Code difference |
|---|---|---|
| Direct | Your code, the driver and the browser on one machine. | `new ChromeDriver()`, `webdriver.Chrome()` |
| Remote | Your code on one machine, driver and browser on another, reached through Selenium Server or RemoteWebDriver. | `new RemoteWebDriver(url, options)`, `webdriver.Remote(url, options=...)` |
| Grid | Selenium Grid receives the commands and relays them to the driver on a node machine. | Same as remote, the URL points at the Grid |

Lesson: Local, remote and Grid, plus BiDi

## TIPS AND TRAPS

REMEMBER

The binding changes some locators before they leave your machine. Look for it in the log in the next lesson. That is why the W3C spec only needs five strategies: css selector, link text, partial link text, tag name and xpath.

Lesson: The W3C WebDriver protocol

REMEMBER

You will never write raw HTTP in real tests. The point is to see that nothing magic happens: Selenium is a polite HTTP client with good manners about waiting, errors and types. When a test fails with a strange message, picture the request that was sent and the error code that came back.

Lesson: See the wire for yourself

## 1.3 Setup, Selenium Manager and your first script

Install Selenium, let Selenium Manager match the driver, and write and tidy up your first real script.

### KEY IDEAS

- **Install** JavaScript: npm install selenium-webdriver (Node 22+).
- **Selenium Manager** Ships with Selenium since 4.6. A fallback that discovers the browser version, resolves, downloads and caches the driver in ~/.cache/selenium, and since 4.11 can download browsers.
- **Driver mismatch** An old driver on PATH or given by hand gives session not created. Delete it and let Selenium Manager match the driver.
- **Eight components** Start session, act on browser, ask the browser, wait strategy, find element, act on element, ask the element, end session.

### CODE

**The eight basic components of a Selenium script**

| | |
|---|---|
| `const { Builder, Browser, By } = require('selenium-webdriver');` | Builder makes drivers, Browser names browsers, By builds locators. |
| `const driver = await new Builder().forBrowser(Browser.CHROME).build();` | 1. Start the session. Chrome opens with a fresh profile. |
| `await driver.get('https://www.selenium.dev/selenium/web/web-form.html');` | 2. Take action on the browser: go to the practice form. |
| `const title = await driver.getTitle();` | 3. Request browser information: the page title. |
| `await driver.manage().setTimeouts({ implicit: 500 });` | 4. Establish a waiting strategy. JavaScript takes milliseconds. |
| `const textBox = await driver.findElement(By.name('my-text'));` | 5. Find an element by its name attribute. |
| `const submitButton = await driver.findElement(By.css('button'));` | 5. Find the first button with a CSS selector. |
| `await textBox.sendKeys('Selenium');` | 6. Take action on an element: type into the box. |
| `await submitButton.click();` | 6. Click Submit. The browser loads the next page. |
| `const message = await driver.findElement(By.id('message'));` | Find the message on the new page. |
| `const value = await message.getText();` | 7. Request element information: its visible text. |
| `await driver.quit();` | 8. End the session. The browser and the driver both close. |

Lesson: The eight steps of a first script

**Quit even when something fails**

| | |
|---|---|
| `const { Builder, Browser, By, error } = require('selenium-webdriver');` | error holds the exception classes. |
| `try {` | Everything that might fail goes inside try. |
| `await driver.findElement(By.id('no-such-id'));` | This element does not exist, so the promise rejects. |
| `if (!(e instanceof error.NoSuchElementError)) throw e;` | Only swallow the error we expected. Anything else is a real bug. |
| `} finally {` | finally runs whether the try worked or failed. |

| | |
|---|---|
| `await driver.quit();` | So the browser always closes. |

Lesson: Always end the session

## TABLES

### Table: language · minimum version for selenium 4.51 · how you add selenium

| Language | Minimum version for Selenium 4.51 | How you add Selenium |
|---|---|---|
| JavaScript | Node.js 22 | `npm install selenium-webdriver` |

Lesson: Install Selenium for your language

### Table: setting · default · what it does

| Setting | Default | What it does |
|---|---|---|
| `cache-path` | `~/.cache/selenium` | Where drivers, browsers and metadata are stored |
| `ttl` | 3600 seconds | How long resolved versions are trusted before asking the vendor again |
| `offline` | false | No network requests or downloads at all |
| `avoid-stats` | false | Selenium Manager sends anonymous usage numbers to Plausible once a day. Set SE_AVOID_STATS=true to stop it. |
| `proxy` | none | An HTTP proxy for the downloads |

Lesson: Selenium Manager finds the driver

### Table: step · what it means · in this example

| Step | What it means | In this example |
|---|---|---|
| 1. Start the session | Create a driver. A browser opens. | Chrome |
| 2. Take action on the browser | Navigate, go back, resize and so on. | Open the web form |
| 3. Request browser information | Title, address, window handles, cookies. | Read the title |
| 4. Establish a waiting strategy | Make sure an element is present before you look for it. | Implicit wait of 500 ms |
| 5. Find an element | Most commands are about elements. | The text box and the button |
| 6. Take action on an element | Type, click, clear. | Type Selenium, click Submit |
| 7. Request element information | Text, attributes, size, state. | Read the message |
| 8. End the session | Quit the driver and close the browser. | quit |

Lesson: The eight steps of a first script

### Table: quit · close

| | quit | close |
|---|---|---|
| Closes | Every window and tab of the session | Only the current window or tab |
| Driver process | Stopped | Keeps running while other windows are open |
| After it | The driver object cannot be used again | Switch to another window to carry on |

Lesson: Always end the session

## TIPS AND TRAPS

REMEMBER

Use a virtual environment in Python and a package.json in JavaScript. Each project then pins its own Selenium version, so upgrading one project never breaks another.

Lesson: Install Selenium for your language

COMMON MISTAKE If you pass a driver path yourself, Selenium trusts you and never calls Selenium Manager. The documentation says that with Selenium 4.6 or newer you should not need to set a driver location at all. Delete old drivers from PATH and let Selenium Manager do the job.

Lesson: Selenium Manager finds the driver

COMMON MISTAKE The documentation itself warns that an implicit wait is rarely the best solution. It is used here because it is the easiest to show. Chapter 4.1 replaces it with explicit waits.

Lesson: The eight steps of a first script

REMEMBER In a test framework the setup and teardown hooks do this job: start the driver before each test and quit it after each test, pass or fail. Chapter 5.1 shows the hooks in JUnit 5, pytest and Mocha.

Lesson: Always end the session

# STEP 2 Control the browser

## 2.1 Sessions, options and timeouts

Options and capabilities, page load strategy, the three session timeouts and the driver service, shown with real runs.

### KEY IDEAS

- **Options** Since Selenium 4, describe sessions with Options classes. They become W3C capabilities, and browser specific ones carry a prefix such as goog:chromeOptions.
- **Page load strategy** normal waits for complete (load), eager for interactive (DOMContentLoaded), none for the first download. It covers navigation by address, not clicks.
- **Default timeouts** Implicit 0, page load 300,000 ms, script 30,000 ms.
- **Service** Starts and stops a local driver: file, port, arguments and logs. Not used for remote sessions.
- **Granted capabilities** Read them after New Session to learn the real browser version, platform and defaults such as dismiss and notify.

### CODE

**Ask for a session with options, then read what the browser agreed to**

| | |
|---|---|
| `const chrome = require('selenium-webdriver/chrome');` | The Chrome module holds chrome.Options. |
| `const options = new chrome.Options();` | An Options object describes the session you want. |
| `options.addArguments('--headless=new', '--window-size=1280,800');` | Chrome command line switches: no window, and a fixed size. |
| `options.setPageLoadStrategy('eager');` | A W3C capability: get() returns at DOMContentLoaded. |
| `const driver = await new Builder().forBrowser(Browser.CHROME).setChromeOptions(options).build();` | New Session is sent with these capabilities. |
| `const caps = await driver.getCapabilities();` | The capabilities the driver actually granted. |
| `console.log(key + ': ' + caps.get(key));` | caps.get reads one capability by name. |

Lesson: Options and capabilities

**Read the three session timeouts, then trip one on purpose**

| | |
|---|---|
| `console.log(await driver.manage().getTimeouts());` | One object with all three timeouts, in milliseconds. |
| `await driver.manage().setTimeouts({ script: 1000 });` | Allow scripts only one second from now on. |
| `await driver.executeAsyncScript('/* never calls its callback */');` | An async script must call its callback. This one never does. |
| `if (!(e instanceof error.ScriptTimeoutError)) throw e;` | The W3C error script timeout becomes ScriptTimeoutError. |

Lesson: The three session timeouts

### Start the driver on a port you choose and ask if it is ready

| | |
|---|---|
| `const service = new chrome.ServiceBuilder().setPort(9600);` | A ServiceBuilder describes how to start chromedriver: here, on port 9600. |
| `const driver = await new Builder().forBrowser(Browser.CHROME).setChromeService(service).build();` | Selenium starts chromedriver on that port, then opens a session. |
| `const reply = await fetch('http://localhost:9600/status');` | GET /status is a W3C command that needs no session. fetch is built into Node. |
| `console.log('ready: ' + status.ready);` | The driver says whether it can start new sessions. |
| `await driver.quit();` | Ends the session and stops the service. |

Lesson: The driver service

## NUMBERS TO REMEMBER

### Implicit wait

0

How long Find Element keeps looking before it gives up with no such element. Zero means it checks once.

Lesson: The three session timeouts

### Page load

Exam favourite

300,000 ms

Five minutes. How long a navigation may take before a timeout error.

Lesson: The three session timeouts

### Script

30,000 ms

Thirty seconds. How long an injected script may run before the script timeout error.

Lesson: The three session timeouts

## TABLES

### Table: capability · what it controls · default

| Capability | What it controls | Default |
|---|---|---|
| `browserName` | Which browser. Set for you by each Options class. | set by the class |
| `browserVersion` | Which version. Selenium Manager can download it if missing. | whatever is installed |
| `platformName` | The operating system at the remote end. Matters most on Grid and in the cloud. | the local OS |
| `pageLoadStrategy` | When navigation commands return: normal, eager or none. | normal |
| `acceptInsecureCerts` | Trust expired or self signed TLS certificates. | false |
| `timeouts` | Script, page load and implicit wait. | 30 s, 300 s, 0 |
| `unhandledPromptBehavior` | What happens to an alert your code did not handle. | dismiss and notify |
| `proxy` | Send browser traffic through a proxy server. | none |
| `setWindowRect` | Whether the browser supports resizing and moving windows. | reported by the driver |
| `strictFileInteractability` | Strict checks for hidden file inputs. | off |

Lesson: Options and capabilities

### Table: chrome switch · what it does

| Chrome switch | What it does |
|---|---|
| `--headless=new` | No visible window. Common on build servers. |
| `--start-maximized` | Open the window maximised. |
| `--window-size=1280,800` | A fixed window size, so pages lay out the same on every machine. |
| `--user-data-dir=...` | Use a chosen profile folder instead of a fresh temporary one. |

Lesson: Options and capabilities

### Table: strategy · waits until readystate is · browser event · use it when

| Strategy | Waits until readyState is | Browser event | Use it when |
|---|---|---|---|
| **normal** (default) | complete | load: every image, font and style sheet is in | You want the safest behaviour |
| **eager** | interactive | DOMContentLoaded: the HTML is parsed, images may still load | Large images or ads slow the page and the test does not need them |
| **none** | any | only the first download of the page | You control every wait yourself |

Lesson: Page load strategy

### Table: language · set the page load timeout to 30 s

| Language | Set the page load timeout to 30 s |
|---|---|
| JavaScript | `await driver.manage().setTimeouts({{ pageLoad: 30000 }})` |

Lesson: The three session timeouts

### Table: service setting · why you would change it

| Service setting | Why you would change it |
|---|---|
| Driver location | Only with Selenium older than 4.6 or a special setup. Otherwise let Selenium Manager find it. |
| Port | A firewall allows only certain ports, or you want to talk to the driver yourself. |
| Log output and level | Driver logs are thrown away unless you send them somewhere. Levels are ALL, DEBUG, INFO, WARNING, SEVERE and OFF. |
| Arguments | Pass chromedriver switches, such as appending to a log file. |

Lesson: The driver service

## TIPS AND TRAPS

REMEMBER Each Options class can also take the binary path of a Chromium based browser, extensions, and excludeSwitches to turn off a default switch of chromedriver. The documentation lists them on the Chrome page. Use them only when you need them, because every extra setting is one more difference from what real users run.

Lesson: Options and capabilities

COMMON MISTAKE Switching to eager or none to make tests faster is a trade. Every element that arrives later now needs an explicit wait, or the test becomes flaky. Chapter 4.1 shows how to write those waits.

Lesson: Page load strategy

IN THE EXAM These timeouts belong to the browser session. Explicit waits such as WebDriverWait are different: they run in your code and poll the browser. Mixing an implicit wait with explicit waits can give surprising wait times, which chapter 4.1 explains.

Lesson: The three session timeouts

## 2.2 Navigation, window size, screenshots and PDF

Navigation and history, title and URL, window size and position, screenshots and printing pages to PDF.

### KEY IDEAS

- **Navigation** get or navigate().to, back, forward, refresh.
- **Where am I** Title and current URL. Check them when a test depends on a redirect or on history, because back on the first page raises no error.
- **Window rect** Set and get size and position. An exact size is more portable than maximize, which needs a window manager.
- **Screenshots** Take Screenshot (viewport) and Take Element Screenshot return base64 PNG. Save one when a test fails, before quit.
- **Print Page** Returns a base64 PDF. Use PrintOptions for orientation, ranges, size, margins and scale. Chromium needs headless.

### CODE

#### Go somewhere, go back, go forward, reload

| | |
|---|---|
| `await driver.get('https://www.selenium.dev/selenium/web/web-form.html');` | Navigate To. Waits for the page load strategy. |
| `await driver.navigate().to('https://www.selenium.dev/selenium/web/xhtmlTest.html');` | The long form of get. |
| `await driver.navigate().back();` | Press the Back button. |
| `await driver.navigate().forward();` | Press the Forward button. |
| `await driver.navigate().refresh();` | Reload the current page. |

Lesson: Navigate and ask where you are

#### Set the window size and position, then read them back

| | |
|---|---|
| `const window = driver.manage().window();` | Window commands live under manage().window(). |
| `await window.setRect({ width: 1024, height: 768, x: 40, y: 30 });` | JavaScript sets size and position in one call to Set Window Rect. |
| `const { width, height, x, y } = await window.getRect();` | Get Window Rect returns all four numbers. |

Lesson: Window size and position

#### Save a screenshot of the page and of one element

| | |
|---|---|
| `const pngSize = file => { const b = fs.readFileSync(file); return [b.readUInt32BE(16), b.readUInt32BE(20)]; };` | A PNG keeps width and height at bytes 16 and 20. |
| `const page = await driver.takeScreenshot();` | Take Screenshot answers with the PNG as base64 text. |
| `fs.writeFileSync('page.png', page, 'base64');` | Decode the base64 and write the bytes to a file. |
| `fs.writeFileSync('button.png', await button.takeScreenshot(), 'base64');` | Take Element Screenshot: just the area of the button. |

Lesson: Screenshots

#### Print the page to a PDF file

| | |
|---|---|
| `const options = new chrome.Options().addArguments('--headless=new');` | Chromium prints to PDF only in headless mode. |
| `const base64 = await driver.printPage({ orientation: 'landscape' });` | The W3C Print Page command answers with base64 text. |
| `const pdf = Buffer.from(base64, 'base64');` | Decode it to the real bytes. |

| | |
|---|---|
| `console.log('page.pdf starts with ' + pdf.subarray(0, 5).toString());` | Every PDF file starts with %PDF-. |

Lesson: Print a page to PDF

## TABLES

### Table: what you want · javascript

| What you want | JavaScript |
|---|---|
| Open an address | `driver.get(url)` or `navigate().to(url)` |
| Back | `navigate().back()` |
| Forward | `navigate().forward()` |
| Reload | `navigate().refresh()` |
| Page title | `getTitle()` |
| Current address | `getCurrentUrl()` |

Lesson: Navigate and ask where you are

### Table: command · what it does

| Command | What it does |
|---|---|
| Set window size | Restores the window if it was maximised, then sets width and height in pixels |
| Set window position | Moves the top left corner to x and y on the screen |
| Get window rect | Width, height, x and y in one answer |
| Maximize | Makes the window fill the screen, without covering the system menus |
| Minimize | Hides the window, usually to the taskbar. The exact result depends on the window manager. Selenium 4 and later. |
| Fullscreen | Fills the whole screen, like pressing F11 |

Lesson: Window size and position

### Table: language · page screenshot to file · element screenshot to file

| Language | Page screenshot to file | Element screenshot to file |
|---|---|---|
| JavaScript | `driver.takeScreenshot()` gives base64 | `element.takeScreenshot()` gives base64 |

Lesson: Screenshots

### Table: printoptions setting · example

| PrintOptions setting | Example |
|---|---|
| Orientation | portrait or landscape |
| Page ranges | "2-4" |
| Page size | A4, Legal and others |
| Margins | top, bottom, left, right |
| Scale | 1.0 is 100 %, 0.25 is 25 % |
| Background | whether background colours and images print |
| Shrink to fit | whether wide content is shrunk to the page |

Lesson: Print a page to PDF

### TIPS AND TRAPS

**COMMON MISTAKE** After refresh or any navigation, the elements you found before belong to the old page. Using them raises stale element reference. Find them again on the new page. Chapter 5.4 covers this error in depth.

Lesson: Navigate and ask where you are

**COMMON MISTAKE** Maximize and minimize need a desktop window manager. On a bare Linux build server they can fail with an unknown error, which is what happened when this page was built. Setting an exact size with Set Window Rect or the --window-size switch works everywhere and gives the same layout on every machine.

Lesson: Window size and position

**REMEMBER** Take a screenshot in the teardown of a failing test, before quit. Name the file after the test, so the report can link to it. Chapter 5.1 wires this into JUnit, pytest and Mocha.

Lesson: Screenshots

## STEP 3 Find and use elements

### 3.1 Locators: how Selenium finds an element

All eight locator strategies, CSS and XPath, relative locators and how to choose a locator that lasts.

### KEY IDEAS

- **Eight strategies** id, name, class name, css selector, xpath, tag name, link text, partial link text. find element returns the first match.
- **Five on the wire** W3C defines css selector, link text, partial link text, tag name and xpath. Bindings turn id, name and class name into CSS.
- **CSS and XPath** CSS: #id, .class, tag[attr="v"], parent > child. XPath adds text matching and moving up to a parent. Prefer relative XPath (//).
- **Relative locators** above, below, toLeftOf, toRightOf and near (50 px), built on getBoundingClientRect. They depend on layout, so check what they return.
- **Preference** Unique stable id, then a short CSS selector, then XPath. Avoid absolute paths, styling classes and generated ids. Consider data-test attributes.

### CODE

**All eight locator strategies on the documentation example page**

| | |
|---|---|
| `async function show(how, e) {` | A helper that prints which element a locator found. |
| `await show('className', await driver.findElement(By.className('information')));` | The first element whose class list contains information. |
| `await show('css', await driver.findElement(By.css('#fname')));` | CSS: # means id, so the input with id fname. |
| `await show('id', await driver.findElement(By.id('lname')));` | The element whose id attribute is lname. |
| `await show('name', await driver.findElement(By.name('newsletter')));` | The element whose name attribute is newsletter. |
| `await show('linkText', await driver.findElement(By.linkText('Selenium Official Page')));` | A link whose whole visible text matches. |
| `await show('partialLinkText', await driver.findElement(By.partialLinkText('Official')));` | A link whose visible text contains Official. |
| `await show('tagName', await driver.findElement(By.tagName('a')));` | The first a element on the page. |

| | |
|---|---|
| `await show('xpath', await driver.findElement(By.xpath("//input[@value='f']")));` | XPath: any input whose value attribute is f. |

Lesson: The eight locator strategies

### Relative locators: find an element by where it sits

| | |
|---|---|
| `const { Builder, Browser, By, locateWith } = require('selenium-webdriver');` | locateWith() starts a relative locator. |
| `console.log(how + ' → ' + await e.getAttribute('outerHTML'));` | outerHTML shows the whole tag that was found. |
| `await show('below last name label', await driver.findElement(locateWith(By.tagName('input')).below(By.css("label[for='lname']"))));` | An input below the Last name label. The closest one wins. |
| `await show('above lname', await driver.findElement(locateWith(By.tagName('input')).above(By.id('lname'))));` | An input above the lname box. |
| `await show('right of Male', await driver.findElement(locateWith(By.tagName('input')).toRightOf(By.css("input[value='m']"))));` | An input to the right of the Male radio. |
| `await show('below fname and above the checkbox', await driver.findElement(locateWith(By.tagName('input')).below(By.id('fname')).above(By.name('newsletter'))));` | Chained: both conditions must hold. |
| `await show('near Newsletter label', await driver.findElement(locateWith(By.tagName('input')).near(By.css("label[for='newsletter']"))));` | near means at most 50 pixels away. Look at what it really returned. |
| `await show('left of password', await driver.findElement(locateWith(By.tagName('input')).toLeftOf(By.id('password-field'))));` | On the login page the three inputs sit in one row. |

Lesson: Relative locators

## TABLES

### Table: strategy · finds · example

| Strategy | Finds | Example |
|---|---|---|
| id | The element whose id attribute matches | `lname` |
| name | Elements whose name attribute matches | `newsletter` |
| class name | Elements whose class list contains the value. Compound names such as "a b" are not allowed. | `information` |
| css selector | Elements matching a CSS selector | `#fname` |
| xpath | Elements matching an XPath expression | `//input[@value='f']` |
| tag name | Elements with that tag | `a` |
| link text | Links whose visible text is exactly the value | `Selenium Official Page` |
| partial link text | Links whose visible text contains the value | `Official` |

Lesson: The eight locator strategies

**Table: to find · css selector · xpath**

| To find | CSS selector | XPath |
|---|---|---|
| By id | `#fname` | `//*[@id="fname"]` |
| By class | `.information` | `//*[contains(@class,"information")]` |
| Tag with attribute | `input[name="fname"]` | `//input[@name="fname"]` |
| Attribute starts with | `a[href^="https"]` | `//a[starts-with(@href,"https")]` |
| Attribute contains | `a[href*="selenium"]` | `//a[contains(@href,"selenium")]` |
| Direct child | `form > input` | `//form/input` |
| Any descendant | `form input` | `//form//input` |
| Second matching input | `input:nth-of-type(2)` | `(//input)[2]` |
| By visible text | not possible | `//label[text()="Last name:"]` |
| The parent | not possible | `//input[@id="lname"]/..` |
| Next sibling | `label + input` | `//label[@for="lname"]/following-sibling::input[1]` |

Lesson: CSS selectors and XPath

**Table: method · finds an element that is · note**

| Method | Finds an element that is | Note |
|---|---|---|
| above | higher on the page than the anchor | |
| below | lower on the page than the anchor | |
| toLeftOf / to_left_of | left of the anchor | |
| toRightOf / to_right_of | right of the anchor | |
| near | at most 50 pixels from the anchor | Useful for an input next to its label |

Lesson: Relative locators

**Table: fragile locator · why it breaks · better**

| Fragile locator | Why it breaks | Better |
|---|---|---|
| `/html/body/div[2]/form/input[3]` | Any new div or input shifts the path | `#lname` |
| `.btn.btn-primary.mt-3` | Styling classes change when the design changes | `[data-test="submit"]` |
| `//button[text()="Submit"]` | Breaks when the text is translated or reworded | An id or a data attribute |
| `#ember1234` | Generated ids change on every page load | A stable attribute near it |

Lesson: Choosing a good locator

## TIPS AND TRAPS

REMEMBER find element (singular) returns the first match or raises no such element. find elements (plural) returns every match in page order, or an empty list. Chapter 3.2 covers the difference in detail.

Lesson: The eight locator strategies

COMMON MISTAKE Each relative locator sends the page a script to run, so it is slower than a plain CSS selector. Prefer an id or a short CSS selector, and keep relative locators for elements with no good attributes.

Lesson: Relative locators

IN THE EXAM Keep locators in one place, apart from the steps that use them. The documentation recommends declaring them separately, which is exactly what a page object does. Chapter 5.2 shows the pattern.

Lesson: Choosing a good locator

## 3.2 Finding elements: one, many, scoped and in shadow DOM

Singular and plural finds, scoped searches and the XPath trap, shadow DOM and the active element.

### KEY IDEAS

- **One or many** find element returns the first match or raises no such element. find elements returns all matches or an empty list.
- **Search context** You can search from the driver, from an element or from a shadow root. CSS from an element stays inside it.
- **XPath trap** An XPath starting with // always starts at the document, even when called on an element. Use .// to search below the element.
- **Shadow DOM** Find the host, get its shadow root, then search with a CSS selector. Chrome refuses XPath there.
- **Active element** switchTo().activeElement() returns the focused element. Use it to test keyboard flow such as Tab order.

### CODE

#### One element, all elements, and what happens when nothing matches

| | |
|---|---|
| `const inputs = await driver.findElements(By.tagName('input'));` | Plural: an array of every input, in page order. |
| `for (const e of inputs) {` | for...of with await inside, so the questions are asked one by one. |
| `const none = await driver.findElements(By.id('email'));` | No match: an empty array, never an error. |
| `await driver.findElement(By.id('email'));` | Singular with no match: the promise rejects. |

Lesson: Find one or find many

#### Search inside one element, and the XPath trap

| | |
|---|---|
| `const form = await driver.findElement(By.css('form'));` | First find the parent element. |
| `console.log('inputs in form: ' + (await form.findElements(By.css('input'))).length);` | Called on the form, CSS only looks inside it. |
| `console.log('links via //a: ' + (await form.findElements(By.xpath('//a'))).length);` | The trap: // starts at the top of the document, so the link outside the form is found. |
| `console.log('links via .//a: ' + (await form.findElements(By.xpath('.//a'))).length);` | The dot means start here. Now nothing outside the form counts. |
| `const last = await form.findElement(By.id('lname'));` | Two commands: find the form, then find inside it. |
| `const same = await driver.findElement(By.css('form #lname'));` | The optimised version: one CSS selector, one command. |
| `console.log('same element: ' + await WebElement.equals(last, same));` | Both references point at the same element. |

Lesson: Search inside an element

#### Reach into a shadow root

| | |
|---|---|
| `console.log('from the page: ' + (await driver.findElements(By.css(box))).length);` | The checkbox lives in a shadow tree, so a normal search finds nothing. |
| `const host = await driver.findElement(By.css('custom-checkbox-element'));` | The shadow host: the element the hidden tree is attached to. |
| `const shadowRoot = await host.getShadowRoot();` | Get Element Shadow Root. |
| `const checkbox = await shadowRoot.findElement(By.css(box));` | Search inside the shadow tree. |
| `await shadowRoot.findElement(By.xpath('.//input'));` | XPath inside a shadow root. |
| `console.log('xpath in shadow root: ' + e.message.split('\n')[0]);` | Chrome refuses it. |

Lesson: The shadow DOM

### Which element has the focus?

| | |
|---|---|
| `console.log('at start: ' + await (await driver.switchTo().activeElement()).getTagName());` | Nothing is focused yet, so the active element is the body. |
| `await driver.findElement(By.name('my-text')).click();` | Clicking a text box gives it the focus. |
| `await (await driver.switchTo().activeElement()).sendKeys(Key.TAB);` | Press Tab on whatever has the focus, as a user would. |
| `console.log('after Tab: ' + await (await driver.switchTo().activeElement()).getAttribute('name'));` | The focus moved to the next field. |

Lesson: The active element

## TABLES

### Table: find element (singular) · find elements (plural)

| | find element (singular) | find elements (plural) |
|---|---|---|
| Returns | One element: the first match in page order | A list of every match, in page order |
| No match | Raises no such element | Returns an empty list |
| W3C command | Find Element | Find Elements |
| Use it for | An element that must be there | Counting, looping, or checking that something is absent |

Lesson: Find one or find many

## TIPS AND TRAPS

COMMON MISTAKE find elements never waits for the first element to appear unless an implicit wait is set, and then it returns as soon as at least one matches. An empty list can mean the page is not ready yet. Chapter 4.1 shows how to wait properly.

Lesson: Find one or find many

REMEMBER Searching inside a container is still very useful when the page repeats a pattern: the Delete button inside the third row of a table, or the price inside one product card. Find the row or card first, then the button inside it.

Lesson: Search inside an element

REMEMBER A shadow root can contain another shadow host. Then you repeat the two steps: get that host, get its shadow root, search again. Each level is one more pair of commands.

Lesson: The shadow DOM

# 3.3 Interacting with elements and reading them

Click, type, clear, the checks before every action and their errors, element information, select lists and uploads.

## KEY IDEAS

- **Five commands** click, send keys, clear, submit and select. Prefer clicking the submit button over submit(). send keys appends, so clear first.
- **Built in checks** Before acting the driver scrolls the element into view and checks it is interactable. Errors: element not interactable, invalid element state, element click intercepted.
- **Element information** isDisplayed, isEnabled, isSelected, tag name, rect, CSS value and text. isDisplayed is a JavaScript approximation.
- **Attribute vs property** DOM attribute is the HTML, DOM property is the live value. The classic getAttribute mixes both.
- **Select and upload** Select works only on select elements: by visible text, value or index. Upload by sending the full path to input type="file".

CODE

### Fill in a form: clear, type, tick and click

| | |
|---|---|
| `await driver.manage().setTimeouts({ implicit: 2000 });` | As in chapter 1.3: let finds retry for up to 2 seconds, so the page after Submit has time to load. |
| `await text.sendKeys('first try');` | Element Send Keys: types into the box, adding to what is already there. |
| `await text.clear();` | Element Clear: empties the box. |
| `await text.sendKeys('Selenium');` | Type the real value. |
| `console.log('check 2 before: ' + await check2.isSelected());` | isSelected reads whether a checkbox is ticked. |
| `await check2.click();` | Element Click: ticks it, as a mouse click would. |
| `await driver.findElement(By.id('my-radio-2')).click();` | Clicking the second radio button unselects the first one. |
| `await driver.findElement(By.css('button')).click();` | Click Submit. The documentation prefers this to the submit() method. |

Lesson: Click, type and clear

### The three errors interactions give, made on purpose

| | |
|---|---|
| `async function attempt(what, step) {` | Run one step and report the error it raises, if any. |
| `if (!(e instanceof error.WebDriverError)) throw e;` | Every Selenium error extends WebDriverError. |
| `await attempt('type into disabled', async () => (await driver.findElement(By.name('my-disabled'))).sendKeys('x'));` | A disabled input cannot take keyboard input. |
| `await attempt('clear readonly', async () => (await driver.findElement(By.name('my-readonly'))).clear());` | A readonly input cannot be reset. |
| `await driver.executeScript(` | Add a cookie banner that covers the bottom of the page, as many real sites do. |
| `await attempt('click under banner', async () => (await driver.findElement(By.css('button'))).click());` | The banner sits over the centre of the button. |

Lesson: Click, type and clear

### Ask an element about itself

| | |
|---|---|
| `await text.sendKeys('hello');` | Type, so the live value differs from the HTML. |
| `console.log('disabled box enabled: ' + await driver.findElement(By.name('my-disabled')).isEnabled());` | The disabled input reports false. |
| `const rect = await text.getRect();` | Get Element Rect: position and size in pixels. |
| `console.log('getAttribute(value): ' + await text.getAttribute('value'));` | The classic method: property first, then attribute. |
| `console.log('getDomAttribute(value): ' + await text.getDomAttribute('value'));` | Only what the HTML says. The HTML has no value attribute. |
| `console.log('getProperty(value): ' + await text.getProperty('value'));` | The live JavaScript property: what is typed now. |
| `console.log('getDomAttribute(myprop): ' + await text.getDomAttribute('myprop'));` | A made up attribute in the HTML. |
| `console.log('button colour: ' + await driver.findElement(By.css('button')).getCssValue('color'));` | The computed CSS value. |
| `console.log('heading text: ' + await driver.findElement(By.css('h1')).getText());` | The rendered, visible text. |

Lesson: Read information about an element

### Work with single and multiple select lists

| | |
|---|---|
| `const single = new Select(await driver.findElement(By.name('selectomatic')));` | Wrap a select element in a Select object. |
| `console.log('options: ' + (await single.getOptions()).length);` | getOptions lists every option element. |
| `await single.selectByVisibleText('Four');` | Pick by the text the user sees. |
| `await single.selectByValue('two');` | Pick by the value attribute. |
| `await single.selectByIndex(0);` | Pick by position, counting from 0. |
| `const multi = new Select(await driver.findElement(By.id('multi')));` | A select with the multiple attribute. |
| `console.log('multi starts with: ' + (await multi.getAllSelectedOptions()).length + ' selected');` | Eggs and Sausages are selected in the HTML. |
| `await multi.deselectByValue('eggs');` | Only multiple lists can deselect. |

Lesson: Select lists

### Upload a file without the file dialog

| | |
|---|---|
| `fs.writeFileSync('report.txt', 'quarterly numbers');` | Make a small file to upload. |
| `const input = await driver.findElement(By.name('my-file'));` | The input with type="file". |
| `await input.sendKeys(path.resolve('report.txt'));` | Send the full path. No dialog opens, the file is simply attached. |
| `console.log('browser shows: ' + await input.getProperty('value'));` | Browsers hide the real folder and show a made up folder called fakepath. |

Lesson: File upload

## TABLES

### Table: command · works on · watch out for

| Command | Works on | Watch out for |
|---|---|---|
| click | Any element | It clicks the centre of the element. If something else is on top there, you get element click intercepted. |
| send keys | Text fields and other keyboard interactable elements | It adds to the existing text. Clear first if you want to replace it. |
| clear | Editable, resettable elements: text inputs and content editable | Readonly and disabled inputs give invalid element state. |
| submit | Form elements | In Selenium 4 it runs a script, so the documentation recommends clicking the submit button instead. |
| select | select and option elements | Use the Select class, covered below. |

Lesson: Click, type and clear

**Table: question · javascript**

| Question | JavaScript |
|---|---|
| Is it displayed? | `isDisplayed()` |
| Is it enabled? | `isEnabled()` |
| Is it selected? | `isSelected()` |
| Tag name | `getTagName()` |
| Position and size | `getRect()` |
| Computed CSS | `getCssValue(name)` |
| Visible text | `getText()` |

Lesson: Read information about an element

**Table: method · reads · use it for**

| Method | Reads | Use it for |
|---|---|---|
| get DOM attribute | Only the attribute in the HTML | Checking what the server sent, such as a data-test value |
| get DOM property | Only the live JavaScript property | What is in a field now, whether a box is checked now |
| get attribute (classic) | The property if there is one, otherwise the attribute. Selenium does this with a JavaScript atom | Old code. Prefer the two precise methods. |

Lesson: Read information about an element

**Table: rule from the documentation · what it means for you**

| Rule from the documentation | What it means for you |
|---|---|
| Select only works on select and option elements | Drop downs built from div or li with JavaScript need normal clicks instead |
| Since Selenium 4.5 a Select cannot be created on a disabled select | Check isEnabled first if the list may be disabled |
| Disabled options cannot be selected | Trying raises an error |
| Only multiple lists can deselect | deselect on a single list raises an error |

Lesson: Select lists

### TIPS AND TRAPS

IN THE EXAM — isDisplayed is not defined by the W3C spec, because deciding what a person can see covers too many cases. The documentation explains that Selenium runs a large JavaScript function that makes approximations. Treat it as a good guide, not a perfect one.

Lesson: Read information about an element

REMEMBER — When the test runs on a Grid, the file must reach the remote machine. Chapter 6.1 shows it.

Lesson: File upload

## STEP 4 Synchronise and control

### 4.1 Waiting strategies: implicit, explicit and fluent

Why tests race the page, implicit and explicit waits, expected conditions, custom waits and why not to mix them.

## KEY IDEAS

- **The race** Content added or shown by JavaScript after load is not covered by navigation waits. Finding it too early fails. Do not use fixed sleeps.
- **Implicit wait** A session setting, default 0. Every find keeps looking up to that long and returns as soon as the element appears. It only waits for presence.
- **Explicit wait** A loop with a condition, timeout and polling interval. JavaScript waits forever without a timeout.
- **Expected conditions** presence, visibility, clickable, text present, title contains, staleness. Wait for visibility before typing.
- **Never mix** Implicit plus explicit makes waits unpredictable: 10 s with 15 s can time out after about 20 s. Keep implicit at 0 and use explicit waits.

## CODE

### The race: look for the box straight after the click

| | |
|---|---|
| `await driver.findElement(By.id('adder')).click();` | The page adds a red box, but only 1 second after the click. |
| `await driver.findElement(By.id('box0'));` | Our code asks at once. No wait is set, so the driver checks only once. |
| `console.log('too early: no such element');` | Selenium was faster than the page. |

Lesson: The race between your code and the page

### Implicit wait: every find keeps looking

| | |
|---|---|
| `await driver.manage().setTimeouts({ implicit: 2000 });` | For the whole session: a find may keep looking for up to 2 seconds. |
| `const box = await driver.findElement(By.id('box0'));` | Not there yet, so the driver keeps checking until the box appears. |
| `console.log('found ' + await box.getAttribute('class') + ' after about ' + Math.round(seconds) + ' s');` | Returns as soon as it is found, not after the full 2 seconds. |

Lesson: Implicit waits

### Explicit wait: wait for exactly the condition you need

| | |
|---|---|
| `const revealed = await driver.findElement(By.id('revealed'));` | The input is in the page already, just hidden. |
| `await driver.findElement(By.id('reveal')).click();` | The page shows it 1 second later. |
| `await driver.wait(() ⇒ revealed.isDisplayed(), 2000);` | Keep calling the function until it is true, for at most 2000 ms. JavaScript checks every 200 ms. |
| `await revealed.sendKeys('Displayed');` | Safe now: the input is displayed, so it can take keys. |

Lesson: Explicit waits and expected conditions

### Ready made conditions, and what a timeout looks like

| | |
|---|---|
| `const { Builder, Browser, By, until, error } = require('selenium-webdriver');` | until holds the ready made conditions in JavaScript. |
| `const box = await driver.wait(until.elementLocated(By.id('box0')), 2000);` | A ready made condition. It resolves to the element. |
| `await driver.wait(until.elementIsVisible(field), 2000);` | Present is not enough for typing. Wait until it is visible. |
| `await driver.wait(until.elementLocated(By.id('box1')), 1000, 'the second box never came');` | Nobody clicked again, so box1 never appears. The third argument is our message. |

Lesson: Explicit waits and expected conditions

### A customised wait: poll faster and retry on an error

| | |
|---|---|
| `await driver.wait(async () ⇒ {` | JavaScript has no ignore list, so the condition catches the error itself. |
| `return true;` | Worked: the wait ends. |

| | |
|---|---|
| `if (e instanceof error.ElementNotInteractableError) return false;` | Not yet: the wait tries again. |
| `throw e;` | Any other error is a real problem. |
| `}, 2000, 'the input never became usable', 300);` | Timeout 2000 ms, our message, and poll every 300 ms. |

Lesson: Customising waits, and never mixing them

TABLES

**Table: strength · weakness**

| Strength | Weakness |
|---|---|
| One line covers every find in the session | It only waits for presence. A present element can still be hidden, covered or disabled |
| Finds return as soon as the element appears | Every check that something is absent now takes the full wait |
| Easy to show in a first script | It cannot wait for text, a title, a count or anything that is not a find |

Lesson: Implicit waits

**Table: default · javascript driver.wait**

| Default | JavaScript driver.wait |
|---|---|
| Polling interval | 200 ms |
| Errors ignored while looping | none, catch them yourself |
| Timeout | 0, which means wait forever. Always pass one |
| Error at the end | TimeoutError |

Lesson: Explicit waits and expected conditions

**Table: wait for · javascript**

| Wait for | JavaScript |
|---|---|
| Element exists | `until.elementLocated` |
| Element visible | `until.elementIsVisible` |
| Element clickable | `until.elementIsEnabled` plus visible |
| Text present | `until.elementTextContains` |
| Title contains | `until.titleContains` |
| Element gone stale | `until.stalenessOf` |

Lesson: Explicit waits and expected conditions

## 4.2 Windows, tabs, frames and alerts

Switching windows and tabs, working inside iframes, and handling alerts, confirms and prompts.

KEY IDEAS

- **Windows** Each window or tab has a handle. Save the original, switch to the new one, and switch back after close(). newWindow("tab") opens and switches in one step.
- **Frames** Switch into an iframe by element, name or id, or index before finding inside it. defaultContent goes to the top, parentFrame up one level.
- **Title in frames** Get Title always reports the top level page, even from inside a frame.
- **Alerts** alert, confirm and prompt: getText, accept, dismiss, sendKeys. Wait with alert is present when it appears late.

- **Forgotten pop ups** unhandledPromptBehavior defaults to dismiss and notify: the pop up is closed and the next command raises unexpected alert open.

CODE

### Follow a link into a new window, come back, open a new tab

| | |
|---|---|
| `const original = await driver.getWindowHandle();` | The id of the current window. It stays the same for the whole session. |
| `await driver.findElement(By.name('windowTwo')).click();` | This link has target="_blank", so it opens a new window. |
| `await driver.wait(async () ⇒ (await driver.getAllWindowHandles()).length ⩵ 2, 2000);` | Wait until the second window exists. |
| `for (const handle of await driver.getAllWindowHandles()) {` | Every open window, in the order they were opened. |
| `if (handle ≠ original) await driver.switchTo().window(handle);` | Switch to the one that is not the original. |
| `await driver.close();` | Close only this window. |
| `await driver.switchTo().window(original);` | Switch back, or the next command fails with no such window. |
| `await driver.switchTo().newWindow('tab');` | Selenium 4: open a new tab and switch to it in one step. |
| `await driver.quit();` | Closes every window and tab at once. |

Lesson: Windows and tabs

### Switch into an iframe three ways and back out

| | |
|---|---|
| `console.log('email boxes seen from the top: ' + (await driver.findElements(By.id('email'))).length);` | The form lives inside the iframe, so the top document sees none. |
| `await driver.switchTo().frame(frame);` | 1. Switch with a WebElement: the most flexible way. |
| `await driver.switchTo().defaultContent();` | Leave the frame: back to the top document. |
| `await driver.switchTo().frame(await driver.findElement(By.name('iframe1-name')));` | 2. JavaScript has no switch by name string, so find the frame by its name first. |
| `await driver.switchTo().frame(0);` | 3. Switch with an index: the first frame of the page. |
| `console.log('title inside the frame: ' + await driver.getTitle());` | Get Title always reports the top level page, even from inside a frame. |
| `console.log('heading: ' + await driver.findElement(By.id('iframe_page_heading')).getText());` | Outside elements are reachable again. |

Lesson: Frames and iframes

### Alert, confirm, prompt, a slow alert and one left open

| | |
|---|---|
| `await driver.findElement(By.id('alert')).click();` | The page calls alert('cheese'). |
| `const alert = await driver.wait(until.alertIsPresent(), 2000);` | Wait for the alert, then get a handle on it. |
| `await alert.accept();` | Press OK. |
| `await driver.findElement(By.id('confirm')).click();` | A confirm box: OK or Cancel. |
| `const confirm = await driver.switchTo().alert();` | Switch to it without waiting, because it opens at once. |
| `await confirm.dismiss();` | Press Cancel. |
| `await driver.findElement(By.id('prompt')).click();` | A prompt asks for text. |
| `await prompt.sendKeys('Selenium');` | Type the answer into the prompt. |
| `console.log('page shows: ' + await driver.findElement(By.id('text')).getText());` | The page wrote our answer into a paragraph. |
| `await driver.findElement(By.id('slow-alert')).click();` | This one opens 200 ms later. |
| `const slow = await driver.wait(until.alertIsPresent(), 2000);` | switchTo().alert() now would fail, so wait. |

| | |
|---|---|
| `await driver.findElement(By.id('alert')).click();` | Open one more alert and forget about it. |
| `await driver.getTitle();` | Any command while an alert is open. |
| `console.log('left open: ' + e.message.split('\n')[0]);` | The default behaviour dismisses it and reports an error. |

Lesson: Alerts, confirms and prompts

## TABLES

### Table: what you want · javascript

| What you want | JavaScript |
|---|---|
| Current handle | `getWindowHandle()` |
| All handles | `getAllWindowHandles()` |
| Switch | `switchTo().window(h)` |
| New tab or window | `switchTo().newWindow("tab")` |
| Close current | `close()` |

Lesson: Windows and tabs

### Table: switch by · example · when to use

| Switch by | Example | When to use |
|---|---|---|
| WebElement | `switchTo().frame(element)` | Most flexible. Find the frame with any locator. |
| Name or id | `switchTo().frame("iframe1-name")` | The frame has a stable name or id. If several match, the first one is used. Java and Python accept the string. JavaScript needs the element. |
| Index | `switchTo().frame(0)` | Position in window.frames. Breaks if frames are added. |
| Leave all frames | `switchTo().defaultContent()` | Back to the top document. |
| Up one level | `switchTo().parentFrame()` | For frames inside frames. |

Lesson: Frames and iframes

### Table: pop up · buttons · what you can do

| Pop up | Buttons | What you can do |
|---|---|---|
| alert | OK | Read the text, accept |
| confirm | OK and Cancel | Read the text, accept or dismiss |
| prompt | OK and Cancel, plus a text box | Read the text, send keys, accept or dismiss. Sending keys replaces the placeholder text |

Lesson: Alerts, confirms and prompts

### Table: unhandledpromptbehavior · what happens to a forgotten pop up

| unhandledPromptBehavior | What happens to a forgotten pop up |
|---|---|
| dismiss and notify (default) | Dismissed, and the command raises unexpected alert open |
| accept and notify | Accepted, and the command raises unexpected alert open |
| dismiss | Dismissed silently |
| accept | Accepted silently |
| ignore | Left open, and the command raises unexpected alert open |

Lesson: Alerts, confirms and prompts

### TIPS AND TRAPS

**COMMON MISTAKE** After close() you are still pointing at the closed window. The documentation warns that the next command raises no such window. Always switch back to a handle you saved before.

Lesson: Windows and tabs

**REMEMBER** Recent drivers dismiss beforeunload prompts, the "Leave site?" box, by default. Chapter 2.1 showed where unhandledPromptBehavior is set.

Lesson: Alerts, confirms and prompts

## 4.3 The Actions API: mouse, keyboard and wheel

Mouse, keyboard and wheel actions, and how a chain becomes ticks.

### KEY IDEAS

- **Build then perform** An action chain is sent in one request when perform() is called. Nothing happens before that.
- **Ticks** Each input source has one action per tick. A tick lasts as long as its slowest action.
- **Mouse** doubleClick, contextClick, moveToElement, clickAndHold, dragAndDrop, moveByOffset. The driver remembers the pointer position between chains.
- **Keyboard** keyDown and keyUp for modifiers, sendKeys for text. Use Command on a Mac and Control elsewhere. Release every key you press.
- **Wheel** scrollToElement, scrollByAmount, scrollFromOrigin. Wait for the scroll result, because it can land after perform() returns.

### CODE

**Double click, right click, hover, drag and drop, and move to a point**

| | |
|---|---|
| `await driver.actions().doubleClick(box).perform();` | Build a chain, then perform() sends it to the browser in one request. |
| `await driver.actions().contextClick(box).perform();` | A right click. |
| `await driver.actions().move({ origin: await driver.findElement(By.id('hover')) }).perform();` | Move to the centre of the element. Nothing is clicked. |
| `await driver.actions().dragAndDrop(from, to).perform();` | Move to from, press, move to to, release. |
| `await driver.actions().move({ x: 8, y: 12 }).perform();` | With no origin, x and y are measured from the top left of the viewport. |
| `await driver.actions().move({ x: 13, y: 15, origin: Origin.POINTER }).perform();` | From wherever the pointer is now. The driver remembered it between chains. |

Lesson: How actions work, and the mouse

**Hold Shift, select all with the right modifier, and type into an element**

| | |
|---|---|
| `.keyDown(Key.SHIFT)` | Press Shift and keep holding it. |
| `.sendKeys('abc')` | Goes to the element that has focus. The page focuses this box on load. |
| `.keyUp(Key.SHIFT)` | Let go of Shift. |
| `const cmdCtrl = process.platform === 'darwin' ? Key.COMMAND : Key.CONTROL;` | Select all is Command A on a Mac and Control A elsewhere. |
| `await driver.actions().click(input).sendKeys('Selenium!').perform();` | JavaScript has no send keys to element in actions, so click first, then type. |

Lesson: Keyboard actions

### Scroll to an element, by an amount, and inside a frame

| | |
|---|---|
| `const frame = await driver.findElement(By.tagName('iframe'));` | The page starts with a 5000 px tall block, so the iframe is far below. |
| `await driver.actions().scroll(0, 0, 0, 0, frame).perform();` | scroll(x, y, deltaX, deltaY, origin). With an element as origin and no delta, it scrolls the element into view. |
| `await driver.get('https://www.selenium.dev/selenium/web/scrolling_tests/frame_with_nested_scrolling_frame_out_of_view.html');` | Load it again. A refresh would keep the scroll position. |
| `await driver.actions().scroll(0, 0, 0, 300).perform();` | Like turning the wheel: 300 px down from the top left of the viewport. |
| `await driver.wait(async () => (await driver.executeScript('return window.scrollY')) >= 300, 2000);` | The scroll can land after perform returns, so wait for it. |
| `await driver.actions().scroll(0, 0, 0, 200, inner).perform();` | Turn the wheel with the pointer over the frame, so the frame scrolls. |

Lesson: Scroll wheel actions

## TABLES

### Table: mouse action · javascript

| Mouse action | JavaScript |
|---|---|
| Double click | `doubleClick(el)` |
| Right click | `contextClick(el)` |
| Hover | `move({{origin: el}})` |
| Press and hold | `press()` |
| Drag and drop | `dragAndDrop(a, b)` |
| Point in the viewport | `move({{x, y}})` |
| From the current point | `move({{x, y, origin: Origin.POINTER}})` |
| Release everything | `actions().clear()` |

Lesson: How actions work, and the mouse

### Table: key method · what it does

| Key method | What it does |
|---|---|
| keyDown(key) | Press a key and keep it down. Meant for modifier keys. |
| keyUp(key) | Release it. Forget this and the key stays down for the next chain. |
| sendKeys(text) | Type into whatever has focus, one key down and key up per character. |
| sendKeys(element, text) | Click the element first, then type. Java and Python only. |

Lesson: Keyboard actions

### Table: wheel action · javascript

| Wheel action | JavaScript |
|---|---|
| Into view | `scroll(0, 0, 0, 0, el)` |
| By an amount | `scroll(0, 0, 0, 300)` |
| From an element | `scroll(0, 0, 0, 200, el)` |

Lesson: Scroll wheel actions

### TIPS AND TRAPS

**REMEMBER** A pointer move has a duration. In the bindings used here, Python uses 250 ms by default and JavaScript 100 ms. Some drag libraries ignore a move that takes 0 ms, so if a drag does nothing, try a chain with a longer move.

Lesson: How actions work, and the mouse

**COMMON MISTAKE** Actions type into the focused element. If nothing has focus, the keys go to the page body and the test seems to type into nothing. Element sendKeys focuses the element itself, which is why it is the better choice for plain typing.

Lesson: Keyboard actions

## 4.4 JavaScript execution and cookies

Running JavaScript in the page, async scripts, and working with cookies.

### KEY IDEAS

- **executeScript** The script runs as a function body. Use return to get a value and arguments[n] to read what you pass in. Elements go in and come back as WebElements.
- **Async scripts** executeAsyncScript adds a callback as the last argument. A returned promise is also awaited. The script timeout, 30 s by default, limits both.
- **JavaScript click** It skips the checks a real click makes, so use it only when you know why the real click cannot work.
- **Cookies** Visit the domain first, then add. httpOnly hides a cookie from page scripts. deleteAllCookies gives a clean start.

### CODE

**Run JavaScript, pass an element in, and see what type comes back**

| | |
|---|---|
| `const type = v => v === null ? 'null' : Array.isArray(v) ? 'Array' : typeof v === 'object' ? v.constructor.name : typeof v;` | A readable name for whatever came back. |
| `console.log(script + ' → ' + type(await driver.executeScript(script)));` | Run the script as the body of a function and convert the result. |
| `console.log('arguments: ' + await driver.executeScript("return arguments[0].innerText + ' / ' + arguments[1]", dinner, 7));` | Extra arguments arrive in the script as arguments[0], arguments[1] and so on. |
| `await driver.executeScript('arguments[0].click();', await driver.findElement(By.id('updatediv')));` | A click done by JavaScript. It skips the checks a real click makes. |

Lesson: Run JavaScript in the page

**Asynchronous scripts, promises, the script timeout and a JavaScript error**

| | |
|---|---|
| `const waited = await driver.executeAsyncScript("const done = arguments[arguments.length - 1]; setTimeout(() => done('waited'), 300);");` | Selenium adds a callback as the last argument. The command returns when the script calls it. |
| `console.log('promise: ' + await driver.executeScript("return new Promise(r => setTimeout(() => r('from promise'), 200));"));` | A returned promise is awaited by the browser before it answers. |
| `await driver.manage().setTimeouts({ script: 200 });` | How long a script may run, in ms. The default is 30000. |
| `console.log('too slow: ' + e.message.split('\n')[0]);` | The callback came after 500 ms, the limit was 200 ms. |
| `console.log('broken script: ' + e.message.split('\n')[0]);` | A JavaScript error in the page becomes javascript error. |

Lesson: Asynchronous scripts and the script timeout

### Add, read, protect and delete cookies

| | |
|---|---|
| `await driver.manage().addCookie({ name: 'key', value: 'value' });` | The browser is still on an empty start page with no domain. |
| `await driver.get('https://www.selenium.dev/selenium/web/blank.html');` | Visit the domain first. A small page is enough. |
| `const c = await driver.manage().getCookie('key');` | Read one cookie by name. It comes back as an object. |
| `await driver.manage().addCookie({ name: 'foo', value: 'bar', sameSite: 'Strict' });` | Any other field goes in the same object. |
| `await driver.manage().addCookie({ name: 'secret', value: 's', httpOnly: true });` | httpOnly hides the cookie from page JavaScript. |
| `console.log('missing cookie: ' + e.name);` | JavaScript raises NoSuchCookieError here, where Java and Python return null or None. |
| `await driver.manage().deleteAllCookies();` | A clean start, for example before logging in as another user. |

Lesson: Cookies

## TABLES

### Table: error · javascript

| Error | JavaScript |
|---|---|
| The script took longer than the script timeout | ScriptTimeoutError |
| The script threw an error | JavascriptError |

Lesson: Asynchronous scripts and the script timeout

### Table: field · meaning

| Field | Meaning |
|---|---|
| name, value | Required. The data. |
| domain, path | Where the cookie is sent. They must match the current page. |
| secure | Only sent over https. |
| httpOnly | Hidden from page JavaScript. |
| sameSite | Strict, Lax or None. Whether the cookie goes with requests started by other sites. |
| expiry | Seconds since 1970. With no expiry the cookie lasts for the session. |

Lesson: Cookies

## TIPS AND TRAPS

COMMON MISTAKE A click done with JavaScript always works, even on a covered or disabled element, because it skips every check a user faces. A test that passes with a JavaScript click may be hiding a real bug. Use it only when you know why the real click cannot work.

Lesson: Run JavaScript in the page

REMEMBER The script timeout is 30 seconds by default, as chapter 2.1 showed. It covers both sync and async scripts. A short timeout makes a broken async script fail fast.

Lesson: Asynchronous scripts and the script timeout

# STEP 5 Build a test suite

## 5.1 Test frameworks: JUnit 5, pytest and Mocha

Running Selenium inside JUnit 5, pytest and Mocha, with data-driven tests and screenshots on failure.

### KEY IDEAS

- **Framework jobs** Selenium drives the browser.
- **Fresh browser** Start the driver in the before each step and quit it in the after each step, so tests stay independent and can run in any order.
- **Teardown always runs** @AfterEach, code after yield and afterEach run also when the test failed, so quit belongs there.
- **Data-driven** @ParameterizedTest with @CsvSource, @pytest.mark.parametrize, or a loop around it() in Mocha. One result per row.
- **Screenshot on failure** Take it after the test failed and before quit: Mocha this.currentTest.state.

### CODE

#### A first test class: a fresh browser for every test

| | |
|---|---|
| `describe('Web form', function () {` | describe groups tests. Mocha gives every test a name. |
| `beforeEach(async function () {` | Runs before every test. |
| `it('has the right title', async function () {` | One test. |
| `assert.strictEqual(await driver.getTitle(), 'Web form');` | Node's own assert module. Actual first, then expected. |
| `afterEach(async function () {` | Runs after every test, also when the test failed. |

Lesson: Selenium inside a test framework

#### One test, three rows of data

| | |
|---|---|
| `const rows = [` | Each row is one run of the test. |
| `for (const [user, password, expected] of rows) {` | Mocha has no data annotation. A loop that calls it() once per row does the same job. |
| `const alert = await driver.wait(until.alertIsPresent(), 2000);` | The page answers with an alert. |

Lesson: Data-driven tests

#### A failing test that saves a screenshot before the browser closes

| | |
|---|---|
| `assert.strictEqual(await driver.getTitle(), 'Web form!');` | Wrong on purpose, so the test fails. |
| `if (this.currentTest.state === 'failed') {` | Mocha tells the hook how the test ended. |
| `fs.writeFileSync(name, await driver.takeScreenshot(), 'base64');` | Take the picture before quit. |

Lesson: A screenshot when a test fails

### TABLES

**Table: job · mocha**

| Job | Mocha |
|---|---|
| Before every test | `beforeEach` |
| After every test | `afterEach` |
| Once for the class or file | `before`, `after` |
| Check a value | `assert.strictEqual(actual, expected)` |
| Run it | `mocha` or `npx mocha` |

Lesson: Selenium inside a test framework

**Table: framework · where the screenshot code goes · how it knows the test failed**

| Framework | Where the screenshot code goes | How it knows the test failed |
|---|---|---|
| Mocha | The `afterEach` hook, before `quit()` | `this.currentTest.state` is "failed" |

Lesson: A screenshot when a test fails

### TIPS AND TRAPS

REMEMBER The Selenium documentation recommends a fresh browser for every test. Then no cookie, login or open dialog can leak from one test into the next, and the tests can run in any order or in parallel. It costs a browser start per test, a price that is almost always worth paying.

Lesson: Selenium inside a test framework

REMEMBER Keep the data close to the test while it is small. When it grows, move it to a CSV or JSON file: JUnit has @CsvFileSource, and pytest and Mocha can read any file into the same list.

Lesson: Data-driven tests

## 5.2 Page objects and page components

Page objects, page component objects and the rules for writing them.

### KEY IDEAS

- **Page object** A class that is the interface to one page. Tests call its methods, so a change to the page is fixed in one place.
- **No assertions inside** Assertions belong to the test. The only check in a page object is that the right page is loaded.
- **Return the next page** A method that moves to another page returns that page object. A method that stays may return itself.
- **Components** A page component object models a repeated part, such as a field or a product card, and searches only inside its root element.
- **Waits inside** Put waits in the page object, for example a constructor that waits for its title after a navigation.

### CODE

**Two page objects, one component and three tests**

| | |
|---|---|
| `const FIELDS = By.css('label.form-label');` | Locators live here and nowhere else. |
| `static async at(driver) {` | A constructor cannot await, so an async factory does the page check. |
| `return (await this.driver.findElements(FIELDS)).map(e => new Field(e));` | Wrap every label element in a Field component. |
| `class Field {` | A page component: one label and the control inside it. |

| | |
|---|---|
| `async name() { return (await this.root.getText()).split('\n')[0].trim(); }` | The first line of the label is its name. |
| `control() { return this.root.findElement(By.css('input, textarea, select')); }` | Search inside this component only. |
| `await driver.wait(until.titleIs('Web form - target page'), 3000);` | The page arrives after a navigation, so wait for it instead of checking once. |
| `const form = await WebFormPage.open(driver);` | The test talks to pages, not to locators. |
| `const received = await form.submit();` | submit() moves to another page, so it returns that page object. |
| `assert.strictEqual(await received.message(), 'Received!');` | The assertion stays in the test. |
| `assert.strictEqual(fields.length, 11);` | Eleven labelled controls on this page. |
| `await assert.rejects(WebFormPage.at(driver), /This is not the web form/);` | The check fails loudly. |

Lesson: Write page objects and components

## TABLES

### Table: without page objects · with page objects

| Without page objects | With page objects |
|---|---|
| Locators are copied into every test that uses the page | Each locator is written once, inside the page object |
| A test reads as a list of clicks and CSS selectors | A test reads as what the user does: open the form, fill a field, submit |
| A UI change breaks many tests in many files | A UI change is fixed in one class |
| Waiting and page checks are repeated | The page object waits for its page once, in its constructor |

Lesson: Why page objects

### Table: rule from the documentation · where it shows in the example

| Rule from the documentation | Where it shows in the example |
|---|---|
| The public methods represent the services the page offers | `field`, `fields`, `submit`, `message` |
| Try not to expose the internals of the page | Locators are private. `Field.control()` is private in Java |
| Generally do not make assertions | Every `assert` is in the test file |
| The one allowed check: the right page is loaded | The constructors check the title, or wait for it after a navigation |
| Methods return other page objects, components or themselves | `submit()` returns ReceivedPage, `fill()` returns the Field |
| A page object need not represent an entire page | `Field` represents one labelled control |

Lesson: Write page objects and components

### Table: choice · what the documentation or practice suggests

| Choice | What the documentation or practice suggests |
|---|---|
| Same action, different result | Write two methods, for example loginAsValidUser and loginExpectingError, each returning the page that really follows |
| A part that appears on many pages | A component object such as a header, a menu or a product card, used inside page objects |
| Where waits go | Inside the page object, so tests do not repeat them |
| Base class | Shared code such as the driver field and a wait helper. Keep it small |

Lesson: Design choices and the limits of page objects

### TIPS AND TRAPS

REMEMBER

ReceivedPage waits for its title, while WebFormPage checks the title once. The web form is opened with get(), which returns after the page loads. The received page appears after a click, and chapter 3.3 showed that the next command can run before the new page is there.

Lesson: Write page objects and components

## 5.3 Good practices and things to avoid

Test design, encouraged practices and the things the documentation says to avoid.

### KEY IDEAS

- **Need a browser?** Ask first whether a lighter test can do the job. Browser tests are the most expensive kind.
- **Short and focused** Set up the data, perform a few actions, evaluate the result. Many small independent tests beat one grand journey.
- **Clean start** A new driver starts with a new profile. Use a fresh browser per test and never share test data.
- **Prepare state outside the UI** Log in and create data through APIs, then set cookies, instead of clicking through forms.
- **Avoid** CAPTCHAs, download progress, status codes, third party logins, test order, performance numbers, spidering and 2FA are jobs for other tools or test hooks.

### CODE

**A new driver starts with a clean profile**

| | |
|---|---|
| `await first.manage().addCookie({ name: 'cart', value: '3 items' });` | State a first test leaves behind: a cookie |
| `await first.executeScript("localStorage.setItem('theme', 'dark')");` | and a value in local storage. |
| `const second = await new Builder().forBrowser(Browser.CHROME).build();` | A new driver gets a new, empty profile by default. |

Lesson: Encouraged practices

**Prepare the state without clicking through the user interface**

| | |
|---|---|
| `const session = 'abc123';` | In a real suite this value comes from a login API call, not from the login form. |
| `await driver.get('https://www.selenium.dev/selenium/web/blank.html');` | A small page on the domain, only to be allowed to set the cookie. |
| `await driver.get('https://www.selenium.dev/selenium/web/cookies.html');` | Now open the page under test. It reads its cookies on load. |

Lesson: Encouraged practices

## TABLES

### Table: testing type · question it answers · selenium fits?

| Testing type | Question it answers | Selenium fits? |
|---|---|---|
| Acceptance | Are we building the right product? | Yes, by simulating what the user expects |
| Functional | Are we building the product right? | Yes |
| Integration | Do the modules work together, for example order plus payment? | Yes, for flows through the UI |
| System | Does the whole product work end to end, in an environment like production? | Yes, a few journeys |
| Performance (load and stress) | How fast is it, and does it survive a peak? | Not as the measuring tool. The documentation points to tools such as JMeter |
| Regression | Did a change break what worked? | Yes, by running the existing tests again |

Lesson: The shape of a good browser test

### Table: practice · what the documentation says

| Practice | What the documentation says |
|---|---|
| Avoid sharing state | Do not share test data, clean up stale data, and create a new WebDriver per test |
| Test independency | Write each test as its own unit, not relying on another test to complete |
| Mock external services | Removing dependencies on external services improves speed and stability |
| Improved reporting | Selenium does not report. Use the reports of your test framework, such as xUnit XML or HTML |
| Locators | Unique ids first, then a well written CSS selector. XPath is flexible but harder to debug and often slower |
| Page objects and a fluent API | Chapter 5.2. Methods can return the page for chained calls |
| Domain specific language | Business level methods on top of page objects |

Lesson: Encouraged practices

### Table: avoid · why · do instead

| Avoid | Why | Do instead |
|---|---|---|
| CAPTCHAs | They are built to stop automation | Disable them in the test environment, or add a hook that lets tests bypass them |
| File downloads | The API does not expose download progress | Find the link with Selenium and fetch it with an HTTP library such as curl, using any cookies it needs |
| HTTP status codes | WebDriver acts like a user, and a status code is rarely the useful detail of a failure | Check what the user sees, such as an error page. Chapter 6.2 shows BiDi network events if you really need codes |
| Logging in to Gmail or Facebook | Against their terms, slow and unreliable | Use the APIs or test account tools they provide |
| Test dependency | Tests must run in any order | Make each test set up its own state |
| Performance testing | Browser start, servers and third parties add noise to every number | Use performance tools, and keep Selenium for function |
| Link spidering | A browser takes seconds per page | Use curl or a library such as BeautifulSoup |
| Two factor authentication | Automating it is hard and less secure | A test only token, or 2FA turned off for test users, test IPs or the test environment |

Lesson: Things to avoid

### TIPS AND TRAPS

REMEMBER Every row has the same shape: Selenium is a tool for what a user does in a browser. When the job is not that, a lighter tool is faster and more reliable.

Lesson: Things to avoid

## 5.4 Troubleshooting: errors, stale elements and logs

Common errors, stale elements and logging in Java, Python and JavaScript.

### KEY IDEAS

- **Error names** Each W3C error has a class per language. Read the name, then the message, then your line in the trace.
- **Stale element** The node behind a reference was replaced, for example by a refresh. Find the element again.
- **Context** An element found in one frame or window is unknown in another. Switch back before using it.
- **Client log** JavaScript 4.51: driver.http.Executor at FINER.
- **Driver log** ChromeDriver writes COMMAND and RESPONSE lines when its service is given a log file and a verbose level.

### CODE

#### Make an element reference fail three ways, and fix each one

| | |
|---|---|
| `let box = await driver.findElement(By.name('my-text'));` | The driver gives back a reference to this exact node. |
| `await driver.navigate().refresh();` | The page is rebuilt. The old node is gone, a new one with the same name takes its place. |
| `box = await driver.findElement(By.name('my-text'));` | The fix: find it again. |
| `const email = await driver.findElement(By.id('email'));` | Found inside the frame. |
| `console.log('2. outside its frame: ' + e.name);` | The reference is fine, but this context does not know it. |
| `await driver.switchTo().frame(frame);` | The fix: switch back to the context where it was found. |
| `await driver.get('https://www.selenium.dev/selenium/web/blank.html');` | Leave the page completely. |
| `console.log('3. after leaving the page: ' + e.name);` | Go back to the page and find it again. |

Lesson: Stale elements and the element store

#### Turn on the client log and see every command

| | |
|---|---|
| `const http = logging.getLogger('driver.http.Executor');` | In selenium-webdriver 4.51 the HTTP lines come from this logger. |
| `http.setLevel(logging.Level.FINER);` | Request lines are written at FINER. |
| `http.addHandler(entry ⇒ {` | A tiny handler that prints one line per request. |
| `const m = /^\[[\w.]+\] >>> (\w+) (\S+)$/.exec(entry.message);` | Each message starts with the logger name in square brackets. |
| `if (m) console.log(m[1] + ' ' + m[2].replace(/\/session\/[0-9a-f]{32}/, '/session/{id}'));` | Hide the session id, which changes every run. |

Lesson: Logs: what the client sent and what the driver did

#### Ask ChromeDriver for its own log

| | |
|---|---|
| `const service = new chrome.ServiceBuilder().loggingTo('chromedriver.log').enableVerboseLogging();` | Where ChromeDriver writes its log, with full detail. |
| `if (line.includes('] COMMAND ')) console.log(line.split('] COMMAND ')[1].split(' ')[0]);` | Print the name of each command the driver received. |

Lesson: Logs: what the client sent and what the driver did

## TABLES

### Table: w3c error · javascript · usual cause and fix

| W3C error | JavaScript | Usual cause and fix |
|---|---|---|
| no such element | NoSuchElementError | Wrong place, wrong time or a changed locator. Check the page, add an explicit wait, update the locator. |
| stale element reference | StaleElementReferenceError | The node was replaced, for example by a refresh or a re render. Find it again. |
| element click intercepted | ElementClickInterceptedError | Something covers the element: a sticky bar, a modal, an animation. Wait for it to go, or scroll. |
| element not interactable | ElementNotInteractableError | Hidden, zero size or not ready. Wait until visible and enabled. |
| invalid selector | InvalidSelectorError | The CSS or XPath is broken, or an XPath was passed to a CSS locator. |
| timeout | TimeoutError | An explicit wait ran out. The message says which condition. |
| script timeout | ScriptTimeoutError | A script took longer than the script timeout. |
| unexpected alert open | UnexpectedAlertOpenError | An alert was left open. Handle it where it opens. |
| no such window | NoSuchWindowError | The window was closed and the test did not switch back. |
| invalid session id | NoSuchSessionError | The session was quit or crashed. Look for a quit earlier in the test. |
| session not created | SessionNotCreatedError | Driver and browser versions do not match, or the browser could not start. |

Lesson: Read an error the right way

### Table: what happened · error in this run · fix

| What happened | Error in this run | Fix |
|---|---|---|
| Refresh, re render, list re sorted | stale element reference | Find the element again just before you use it |
| Switched to another frame or window | no such element | Switch back to the context where it was found |
| Navigated to another page | no such element | Go back to the page and find it again |

Lesson: Stale elements and the element store

### Table: language · logger · default level · level that shows requests

| Language | Logger | Default level | Level that shows requests |
|---|---|---|---|
| JavaScript | selenium-webdriver logging, driver.http.Executor in 4.51 | OFF until set | FINER |

Lesson: Logs: what the client sent and what the driver did

## TIPS AND TRAPS

REMEMBER Search the error name together with the first line of the message. Ignore the long list of numbers under Stacktrace in Chrome messages. Those are addresses inside ChromeDriver, not your code.

Lesson: Read an error the right way

REMEMBER Page objects that find elements inside their methods, as in chapter 5.2, rarely meet stale elements, because they look the element up fresh each time.

Lesson: Stale elements and the element store

COMMON MISTAKE The documentation shows the JavaScript logger as getLogger("webdriver"). In selenium-webdriver 4.51 that name printed nothing in our run, while driver.http and driver.http.Executor printed every request.

Lesson: Logs: what the client sent and what the driver did

# STEP 6 Scale out with Grid

## 6.1 Selenium Grid 4

Selenium Grid 4 components, roles, remote sessions and parallel runs.

### KEY IDEAS

- **Why a Grid** Parallel runs on many browsers, versions and systems, and shorter suites: tests × average time ÷ nodes.
- **Components** Router (4444), New Session Queue, Distributor, Session Map, Node (5555) and Event Bus (4442, 4443).
- **Roles** Standalone for one machine, Hub and Node for most Grids, Distributed for very large ones.
- **Remote driver** Only the driver creation changes. se:name adds metadata, se:cdp gives DevTools through the Grid, and a local file detector sends upload files to the node.
- **Queue and quit** New sessions wait in the queue for a free slot. Always quit, or the slot stays busy until the session timeout.

### CODE

#### Ask the Grid if it is ready and what it offers

```
const res = await fetch('http://localhost:4444/status');
```

The same endpoint a driver has, now answered by the Grid.

```
console.log('slot: ' + slot.stereotype.browserName);
```

Each slot can hold one session of this browser.

Lesson: When and why to use a Grid

#### Three tests, two slots: watch the queue

```
const driver = await new Builder().forBrowser(Browser.CHROME).usingServer('http://localhost:4444').build();
```

Resolves when the Grid has given this test a slot.

```
await new Promise(r => setTimeout(r, 3000));
```

Pretend the test takes 3 seconds.

```
await driver.quit();
```

Frees the slot for the next request in the queue.

```
const times = (await Promise.all([oneTest(), oneTest(), oneTest()])).sort((a, b) => a - b);
```

Three tests start at the same moment.

```
console.log('got a browser after: ' + JSON.stringify(times.map(t => t + ' s')));
```

Two at once, the third only after a slot was free.

Lesson: Components, roles and ports

#### Run a test on the Grid with a name and a file upload

```
const options = new chrome.Options();
```

Options say which browser you want. The Grid picks a node that has it.

```
options.set('se:name', 'upload on the grid');
```

se: capabilities are Grid metadata. se:name shows in the Grid UI.

```
const driver = await new Builder().forBrowser(Browser.CHROME).setChromeOptions(options).usingServer('http://localhost:4444').build();
```

The only change from a local test: a server URL.

```
console.log(k + ': ' + (k === 'se:cdp' ? v.replace(id, '{id}') : v));
```

se:cdp is a DevTools address that goes through the Grid.

```
driver.setFileDetector(new remote.FileDetector());
```

Sends the local file to the node, which may be another machine.

```
console.log('attached: ' + await driver.executeScript('return arguments[0].files[0].name', upload));
```

The browser on the node has the file.

Lesson: Remote sessions, metadata and files

## NUMBERS TO REMEMBER

**Suite time on a Grid** Exam favourite

$$\text{total time} = \frac{\text{number of tests} \times \text{average test time}}{\text{number of nodes}}$$

From the documentation: 15 tests of 45 s take 11 min 15 s on one node, 2 min 15 s on 5 nodes and 45 s on 15 nodes.

Lesson: When and why to use a Grid

## TABLES

**Table: component · job · default port**

| Component | Job | Default port |
|---|---|---|
| Router | Entry point for every request. New sessions go to the queue. Commands for existing sessions go straight to the right node, found through the Session Map | 4444 |
| New Session Queue | Holds new session requests in first in, first out order, with a timeout | 5559 |
| Distributor | Knows every node and its slots. Takes requests from the queue, finds a matching free slot and creates the session | 5553 |
| Session Map | Remembers which node runs which session id | 5556 |
| Node | Runs the browsers. By default one slot per CPU for Chromium browsers and Firefox, one for Safari | 5555 |
| Event Bus | Messages between the other components, cheaper than HTTP. Started first in a distributed Grid | 4442, 4443 and 5557 |

Lesson: Components, roles and ports

**Table: role · command · use it for**

| Role | Command | Use it for |
|---|---|---|
| Standalone | `java -jar selenium-server-<version>.jar standalone` | All components in one process on one machine. Local debugging, quick suites, CI |
| Hub and Node | `... hub` and `... node --hub http://<hub-ip>:4444` | The most used role. Combine machines with different systems and browsers behind one address |
| Distributed | `... event-bus`, `sessionqueue`, `sessions`, `distributor`, `router`, `node` | Large Grids with more than 100 nodes, each component scaled on its own |

Lesson: Components, roles and ports

**Table: language · remote driver · file detector**

| Language | Remote driver | File detector |
|---|---|---|
| JavaScript | `new Builder().usingServer(gridUrl)` | `setFileDetector(new remote.FileDetector())` |

Lesson: Remote sessions, metadata and files

## TIPS AND TRAPS

REMEMBER

A node registers by sending a message on the Event Bus, and the Distributor then calls it over HTTP to confirm. When hub and node are on different machines, the Event Bus ports 4442 and 4443 and the node port must be reachable.

Lesson: Components, roles and ports

## 6.2 WebDriver BiDi, CDP and Grid features

WebDriver BiDi events, Chrome DevTools Protocol commands and Grid GraphQL.

### KEY IDEAS

- **BiDi** A W3C protocol with a WebSocket, so the browser can push events. Turn it on with webSocketUrl, then subscribe to console and page errors.
- **Events, not polling** Handlers are called when events arrive. Wait for them, because they come after the action that caused them.
- **CDP** Chrome and Edge only, no stability promise, the three newest Chrome versions. Use it for Chromium features with no other API.
- **GraphQL** POST a query to /graphql on the Grid for nodes, slots, sessions and the queue size.
- **Grid endpoints** /status, DELETE /session, and drain or remove a node through the Distributor endpoints.

### CODE

#### Hear console messages and page errors as they happen

| | |
|---|---|
| `const driver = await new Builder().forBrowser(Browser.CHROME).setChromeOptions(new chrome.Options().enableBidi()).build();` | Ask for a BiDi WebSocket next to the normal HTTP connection. |
| `const inspector = await LogInspector(driver);` | Subscribes to log events. |
| `await inspector.onConsoleEntry(e ⇒ heard.push(`console ${e.level}: ${e.text}`));` | Called for every console.log, console.error and so on. |
| `await inspector.onJavascriptException(e ⇒ heard.push(`page error: ${e.text}`));` | Called for every uncaught error in the page. |
| `await driver.wait(() ⇒ heard.length ⩵ 3, 5000);` | Events are pushed, not returned, so wait until all three arrived. |

Lesson: WebDriver BiDi: the browser talks back

#### Send Chrome DevTools commands: read the version and change the time zone

| | |
|---|---|
| `const driver = await new Builder().forBrowser(Browser.CHROME).build();` | sendAndGetDevToolsCommand exists on Chrome and Edge drivers only. |
| `const version = await driver.sendAndGetDevToolsCommand('Browser.getVersion', {});` | Any CDP method by name, with an object of parameters. |
| `await driver.sendAndGetDevToolsCommand('Emulation.setTimezoneOverride', { timezoneId: 'Asia/Kolkata' });` | Something WebDriver Classic has no command for. |

Lesson: The Chrome DevTools Protocol

#### Ask the Grid exactly what you need with GraphQL

| | |
|---|---|
| `const res = await fetch('http://localhost:4444/graphql', { method: 'POST', body: JSON.stringify({ query }) });` | GraphQL over HTTP: one POST with the query as a string. |
| `const q = '{ grid { nodeCount, totalSlots, maxSession, sessionCount, sessionQueueSize } }';` | Ask only for these fields. |
| `console.log('busy: ' + JSON.stringify((await ask(q)).grid));` | One session is running now. |
| `console.log('names: ' + JSON.stringify(sessions.map(s ⇒ JSON.parse(s.capabilities)['se:name'])));` | The se:name metadata is in the session capabilities. |

Lesson: Grid features: GraphQL and endpoints

### TABLES

**Table: how to switch it on · javascript**

| How to switch it on | JavaScript |
|---|---|
| Capability | `new chrome.Options().enableBidi()` |
| Console and page errors | `await LogInspector(driver)` |

Lesson: WebDriver BiDi: the browser talks back

**Table: webdriver classic · cdp · webdriver bidi**

| | WebDriver Classic | CDP | WebDriver BiDi |
|---|---|---|---|
| Standard | W3C Recommendation | Chrome only, no stability promise | W3C, in progress |
| Browsers | All | Chrome and Edge | Chrome, Edge and Firefox, growing |
| Style | Request and response over HTTP | Commands and events over WebSocket | Commands and events over WebSocket |
| Use it for | Every normal test step | Chromium features with no other API | Events, logs, network, and the future of Selenium |

Lesson: The Chrome DevTools Protocol

**Table: endpoint · what it does**

| Endpoint | What it does |
|---|---|
| GET /status | Current state of the Grid and every node, with slots and sessions |
| POST /graphql | Ask for exactly the fields you need: grid, nodesInfo, sessionsInfo, session(id) |
| DELETE /session/<id> | End a session and free its slot |
| POST /se/grid/distributor/node/<node-id>/drain | Stop a node after its running sessions end, with no new sessions. Needs the registration secret header |
| DELETE /se/grid/distributor/node/<node-id> | Remove a node from the Distributor. Its running sessions continue |

Lesson: Grid features: GraphQL and endpoints

### TIPS AND TRAPS

REMEMBER The documentation calls out two popular CDP examples with limited value: emulated geolocation, because most sites use the IP address, and overriding device metrics, because Chrome mobile emulation in the options is the better tool.

Lesson: The Chrome DevTools Protocol

## STEP 7 Revise and practise

### 7.1 Revision: predict, debug and design

Predict the output, find the bug, design and sizing questions, a cheat sheet in three languages and a mock quiz.

### KEY IDEAS

- **Predict, then run** The driver does what the protocol says, not what the code seems to say. Offsets add, Shift changes case, a refresh makes references stale.
- **Cause before fix** Name why a test is slow or flaky before changing it. Sleeps, mixed waits and missing quit calls are the usual causes.
- **Layers** Tests, page objects and components, a driver factory and support code. Each change should touch one layer.

- **Size with the formula** Suite time = tests × average time ÷ nodes, with about one CPU and 1 GB of RAM per browser.
- **Know your binding** Time is a Duration in Java, seconds in Python and milliseconds in JavaScript, and a missing cookie gives null, None or an error.

## NUMBERS TO REMEMBER

**Page load timeout**

300,000 ms

Five minutes by default.

Lesson: Cheat sheet in three languages

**Script timeout**

30,000 ms

Thirty seconds by default.

Lesson: Cheat sheet in three languages

## TABLES

**Table: question shape · what it looks like · what earns the marks · lesson**

| Question shape | What it looks like | What earns the marks | Lesson |
|---|---|---|---|
| Predict the output | A short program and the question: what does it print, or which error comes back | Knowing what the driver really does, not what the code seems to say | 2 |
| Find the bug | A test that is slow, flaky or fails, and the question: why, and how do you fix it | Naming the cause first, then a fix that removes it | 3 |
| Design and sizing | Plan a framework, size a Grid, triage a flaky suite | Clear layers, numbers from a formula, a reason for each choice | 4 |
| Write it in another language | The same step in Java, Python or JavaScript | The right method name and the right unit for time | 5 |

Lesson: How Selenium skills are tested

**Table: step · javascript**

| Step | JavaScript |
|---|---|
| Start Chrome | `await new Builder().forBrowser(Browser.CHROME).build()` |
| Open a page | `await driver.get(url)` |
| Find one element | `await driver.findElement(By.id("q"))` |
| Type | `await el.sendKeys("text")` |
| Wait for an element | `await driver.wait(until.elementLocated(By.id("q")), 5000)` |
| Into a frame | `await driver.switchTo().frame(el)` |
| Back to the page | `await driver.switchTo().defaultContent()` |
| Run a script | `await driver.executeScript("return 1")` |
| Add a cookie | `await driver.manage().addCookie({{name: "k", value: "v"}})` |
| Use a Grid | `await new Builder().usingServer("http://localhost:4444").forBrowser(Browser.CHROME).build()` |
| End the session | `await driver.quit()` |

Lesson: Cheat sheet in three languages

## TIPS AND TRAPS

REMEMBER

Use the Revise lessons in order the first time, then use the Quiz and Worked problems tabs as a timed mock test. Each answer says which chapter to reread if you got it wrong.

Lesson: How Selenium skills are tested

COMMON MISTAKE A good bug answer has three parts: what the code really does, why that breaks, and a fix that removes the cause. A retry that hides the failure is not a fix.

Lesson: Find the bug